

AI Usage Note – PS-05 RCA Drafter

AI Usage Note - PS-05 RCA Drafter

Summary

This document describes how AI assistants (Ollama LLM) were used to build the RCA Drafter prototype, what worked well, what needed correction, and the best prompts used.

What AI Helped With

1. **Architecture & Design** ■

- **What AI did:** Generated the initial multiagent architecture design with specialized agents for timeline, log, and code analysis, plus a coordinator and synthesizer
- **Outcome:** Clean separation of concerns with MCP-inspired tools for data extraction and specialized agent prompts
- **Benefit:** Reduced ambiguity about how to delegate tasks and coordinate between agents, enabling parallel execution

2. **Code Generation** ■

- **What AI did:** Generated utility functions for regex-based parsing of timelines, log extraction, and git diff parsing (MCP tools), plus specialized agent classes (timeline, log, code, synthesis, coordinator)
- **Outcome:** Functional parsing tools that handle common patterns (HH:MM timestamps, ERROR log lines, git diff hunks) and agent classes with specialized LLM prompts
- **Benefit:** Saved 2-3 hours of manual regex writing and edge case handling; saved additional time by generating multiple agent classes with clear responsibilities

3. **Structured Output Generation** ■

- **What AI did:** Created the synthesis agent logic that combines insights from specialized agents and generates the final RCA Markdown using MCP tools
- **Outcome:** Consistent, professionally formatted RCA reports following SRE best practices
- **Benefit:** Every report includes required sections (Timeline, Root Cause, Action Items, Prevention) with evidence grounding from all data sources

4. **Flask Web App** ■

- **What AI did:** Built the web form interface and API endpoints for incident data submission
- **Outcome:** User-friendly web UI with real-time feedback and downloadable RCA output
- **Benefit:** Non-technical users can generate RCAs without command-line knowledge

What AI Got Wrong (& How We Fixed It)

1. **Initial LLM Prompt Too Generic**

- **Problem:** First version of the analysis prompt was asking the LLM to generate free-form RCA text
- **Result:** Output was narrative-style instead of structured for easy integration
- **Fix:** Rewrote prompt to require JSON output with specific keys (`primary_root_cause`, `contributing_factors`, etc.)
- **Lesson:** Always constrain LLM output format to what you need programmatically

2. **Missing Error Handling in MCP Tools**

- **Problem:** Parse functions would crash on malformed input (missing timestamps, blank lines)
- **Result:** Fragile parsing that failed on real-world messy data
- **Fix:** Added null checks, default values, and filters for short/empty lines
- **Lesson:** Test parsing tools with intentionally bad data, not just happy paths

3. **Ollama Connection Not Validated**

- **Problem:** App would start but crash when trying to call the LLM if Ollama wasn't running
- **Result:** Confusing errors for new users
- **Fix:** Added `validate_ollama_connection()` method and better error messages
- **Lesson:** Always provide a health check for external dependencies

4. **LLM Sometimes Hallucinated Numbers**

- **Problem:** LLM-generated action items occasionally referenced non-existent log entries or made up statistics
- **Result:** Reduced credibility of RCA
- **Fix:** Added evidence-grounding in the prompt: "Only quote from provided data, never fabricate"
- **Lesson:** Use low temperature (0.3) for factual analysis tasks, and explicitly forbid fabrication

Best Prompts Used

Prompt 1: Multi-Step Agent Loop Orchestration (Successful)

```

You are an expert SRE analyzing a production incident.

INCIDENT: {incident\_title}

TIMELINE EVENTS: {json.dumps(timeline\_events, indent=2)}

ERROR LOGS EXTRACTED: {json.dumps(log\_errors, indent=2)}

RECENT CODE CHANGES: {json.dumps(code\_changes, indent=2)}

CROSS-REFERENCE INSIGHTS: {json.dumps(cross\_ref, indent=2)}

Based on the above data, provide a concise analysis covering:

1. PRIMARY ROOT CAUSE: Identify the single deepest technical cause (not a symptom)
2. CONTRIBUTING FACTORS: List 2-3 factors that made the problem worse
3. EVIDENCE: Quote specific log lines and code changes that confirm the root cause
4. SEVERITY: Classify as P1/P2/P3 based on impact
5. ACTION ITEMS: Generate 4-6 specific, actionable remediation steps

Format your response as structured JSON with these exact keys:

{primary\_root\_cause, contributing\_factors, evidence, severity, action\_items, lessons\_learned}

Respond ONLY with valid JSON, no additional text.

...

**\*\*Why it worked:\*\***

- Provided structured extracted data upfront (no raw log parsing by LLM)
- Constrained output format to JSON for easy programmatic handling
- Explicitly forbidden fabrication ("Quote specific log lines")
- Clear severity classification criteria

## ***Prompt 2: Root Cause Identification (Iteration)***

**\*\*First attempt (Failed):\*\***

...

Analyze these logs and explain what went wrong.

...

**\*\*Result:\*\*** LLM generated vague narrative, not actionable diagnosis

**\*\*Revised version (Successful):\*\***

...

Review the PRIMARY TIMELINE EVENT where failure occurred (marked below).

Cross-reference with ERROR LOGS to find: Was this error caused by (A) recent code change, (B) config change, or (C) environmental factor?

Quote the specific log line that confirms your hypothesis.

Then identify: Is this a SYMPTOM (e.g., "high memory") or a ROOT CAUSE (e.g., "memory leak in code change X")?

Answer: {brief answer}

...

**\*\*Why revision worked:\*\***

- Explicit event correlation (timeline → logs → code)
- Forced distinction between symptom and root cause (common SRE error)
- Required evidence quoting (prevents hallucination)

### **Prompt 3: Action Item Generation (Successful)**

...

For each identified root cause and contributing factor, generate ONE specific action item.

Format:

Action: [specific fix or process improvement]

Priority: [High|Med|Low - based on severity and effort]

Owner: [team/role - e.g., Platform Engineering, SRE]

Constraints:

- Must be specific enough to assign to a person (NOT "improve monitoring")
- Must directly trace back to a root cause above
- Must be doable within 1-2 weeks

...

**\*\*Why it worked:\*\***

- Specificity requirement prevented generic action items
- Traceability requirement kept focus on root causes
- Effort scoping (1-2 weeks) made items realistic

---

## **Temperature & Model Settings Used**

- **\*\*Temperature:\*\*** 0.3 (Low – for deterministic, factual analysis)
- **\*\*Model:\*\*** Ollama Mistral (7B parameters, good balance of speed/quality)
- **\*\*Timeout:\*\*** 60 seconds per LLM call

---

## **What Still Needs Human Review**

1. **\*\*Severity Classification\*\*** – LLM guesses based on error count; human must validate against actual SLA impact
2. **\*\*Action Item Prioritization\*\*** – LLM assigns based on urgency; humans should reprioritize based on resource availability
3. **\*\*Long-term Prevention Steps\*\*** – LLM suggests process improvements; architecture review team must assess feasibility

---

## **Lessons for Future Use**

| Lesson | Applies To | Recommendation |

|-----|-----|-----|

| Constrain LLM output format early | All text generation | Always request JSON or structured format, not free text |

| Validate external dependencies | Integration tasks | Add health checks for APIs, databases, LLM services |

| Test with bad data | Data parsing | Create test cases with malformed/missing fields |

| Ground LLM in evidence | Factual tasks | Provide structured extracted data; forbid fabrication |

| Use low temperature | Deterministic tasks | Set temp  $\leq 0.3$  for analysis; use higher for creative tasks |

| Iterate on prompt clarity | Prompt engineering | Test multiple prompt phrasings; measure output quality |

---

## Example LLM Call Logs

### ***Successful Call***

---

Request: Analyze timeline + logs + code changes

LLM Response:

```
{
 "primary_root_cause": "Database connection pool max size reduced from 50 to 30 in config change",
 "contributing_factors": [
 "Guard clause removed from PaymentService, allowing unbounded queueing",
 "No alerting on pool utilization"
],
 "evidence": {
 "from_logs": "Cannot get a connection, pool error Timeout waiting for idle object",
 "from_code": "database.properties line 2: -maxConnections=50 +maxConnections=30",
 "from_timeline": "Errors began within 2 minutes of config deployment"
 },
 "severity": "P1",
 "action_items": [...]
}
```

Result: ■ Parsed successfully, high confidence RCA generated

---

### ***Failed Call (and Recovery)***

---

Request: Generate action items (old prompt)

LLM Response: "Improve system reliability and monitoring to prevent future issues"

Result: ■ Too generic, not actionable

Recovery:

- Revised prompt to require specific, traceable action items
- Retried with new prompt

Result: ■ Generated "Add pool utilization alerting (>80%) in Prometheus" → Success

...

---

## Conclusion

AI assistance was most valuable for:

1. **\*\*Multiagent architecture design\*\*** – Creating specialized agents with clear responsibilities and communication flows
2. **\*\*Parsing & extraction\*\*** – Handling messy real-world data formats with MCP tools
3. **\*\*Prompt engineering\*\*** – Developing specialized LLM prompts for each agent type
4. **\*\*Template generation\*\*** – Creating structured, professional output via the synthesis agent

AI required human oversight for:

1. **\*\*Agent coordination\*\*** – Ensuring proper data flow between agents and handling edge cases
2. **\*\*Root cause validation\*\*** – Ensuring analysis matches business impact
3. **\*\*Data grounding\*\*** – Preventing LLM hallucinations in agent outputs
4. **\*\*Priority setting\*\*** – Aligning action items with organizational constraints

**\*\*Overall AI contribution:\*\*** Reduced development time from ~4 days to ~16 hours (75% faster) while enabling a more scalable and maintainable architecture.